



Exercises session 5

Reductions, NP-hard, NP-complete

1 Definitions

Definition 1 (Computable functions) A function $f : \Sigma^* \rightarrow \Sigma^*$ is a **polynomial time computable function** if some polynomial time Turing machine exists that halts with just $f(w)$ on its tape, when started with the input $w \in \Sigma^*$.

Definition 2 (Karp reductions) Language A is **polynomial time Karp reducible** to language B , written $A \leq_P B$, if there is a polynomial time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that:

$$w \in A \iff f(w) \in B.$$

Definition 3 (NP) A language L is in NP if there exists a polynomial time verifier V such that for every $w \in \Sigma^*$:

$$w \in L \iff \exists c \text{ with } |c| = \text{poly}(|w|) \text{ and } V(w, c) = 1.$$

The string c is called a *certificate* (or *witness*) for w .

Definition 4 (P) A language L is in P if there exists a deterministic Turing machine that decides L in polynomial time.

Definition 5 (NP-Hardness) A language B is **NP-hard** if for any $A \in NP$ we have $A \leq_P B$.

Definition 6 (NP-Completeness) A language B is **NP-complete** if B is NP-hard and $B \in NP$.

Theorem 1 (Cook-Levin) SAT is NP-complete.

2 Session exercises

1. Show that if L is NP-hard and $L \in P$, then $P = NP$.

Hint: use the definition of NP-hardness together with the fact that polynomial-time reductions compose with polynomial-time algorithms.

2. Prove that if $P = NP$, then every language in P , except $\emptyset$ and Σ^* , is NP-complete.

Hint: you need to show both NP-hardness and membership in NP. For the hardness direction, think carefully about why $\emptyset$ and Σ^ must be excluded.*

3. Let A, B, C be languages such that $A \leq_P B$ and $B \leq_P C$. Show that $A \leq_P C$.

Write the composed reduction function explicitly and verify that it runs in polynomial time.

4. Show that if $A \leq_P B$ and A is NP-hard, then B is NP-hard.

Follow-up: what additional condition do you need on B to conclude that B is NP-complete?

5. For each statement below, prove whether it is true or false.

(a) For every NP-hard problem X , we have $X \leq_P \text{SAT}$.

(b) Let X be an NP-complete problem. If there exists an algorithm to solve X in polynomial time, then SAT can be solved in polynomial time.

6. Let M be a Turing machine and t a number given in unary. Consider the following problem:

$$\text{ACCEPTS@}t = \{\langle M, t \rangle : M \text{ accepts some input in less than } t \text{ steps}\}$$

Prove by definition that $\text{ACCEPTS@}t$ is NP-complete.

Recall that to prove NP-completeness you need to show (i) membership in NP — what is the certificate? — and (ii) NP-hardness — every NPlanguage reduces to it.

P1 1. Show that if L is NP-hard and $L \in P$, then $P = NP$.

Hint: use the definition of NP-hardness together with the fact that polynomial-time reductions compose with polynomial-time algorithms.

Sol
Take arbitrary $A \in NP$; then $A \leq_p L$ (because L is NP-hard).

As $A \leq_p L$, there exists f computable in polynomial time s.t.

$$w \in A \iff f(w) \in L$$

Since $L \in P$, then $\exists$ dl that decides L in P . then we can prove that $A \in P$, because A can be solved by:
(1) take w (2) compute $f(w)$
(3) run dl on $f(w)$. Full procedure is polynomial.

$$\therefore A \in P \implies NP \subseteq P$$

As $P \subseteq NP$; then we conclude

$$P = NP.$$

2. Prove that if $P = NP$, then every language in P , except $\emptyset$ and Σ^* , is NP-complete.

We will assume

$$P = NP$$

then take a language $B \in P$

with $B \neq \emptyset$ and $B \neq \Sigma^*$

then there exists words $w_1, w_2 \in \Sigma^*$
such that

$$w_1 \in B$$

$$w_2 \notin B$$

For our assumption, $B \in NP$

TO PROVE: B is NP-complete

① $B \in NP$

trivial, because $B \in P$

$$P \subseteq NP$$

$$\Rightarrow B \in NP.$$

② B is NP-hard

Lets take $A \in NP$, then as $P = NP$,
A has a polynomial decider M_A .
then we can do the following reduction

INPUT: $\langle w \rangle$

1. run M_A on w
 2. if M_A accepts, output w_1
if M_A rejects, output w_2 .
- } $f(w)$

We now prove that the reduction is correct:

$$w \in A \iff f(w) \in B.$$

$$w \in A \implies f(w) = w_1$$

$$w_1 \in B$$

$$w \notin A \implies f(w) = w_2$$

$$w_2 \notin B //$$

3. Let A, B, C be languages such that $A \leq_P B$ and $B \leq_P C$. Show that $A \leq_P C$.

We have

$A \leq_P B \iff \exists \text{ computable poly-time } f \text{ s.t.}$

$$w \in A \iff f(w) \in B$$

$B \leq_P C \iff \exists \text{ comp. poly-time } g \text{ s.t.}$

$$w \in B \iff g(w) \in C$$

We want to find h s.t.

$$w \in A \iff h(w) \in C$$

take $h = g \circ f$ ($h(w) = g(f(w))$)

Composition is polynomial, so h runs in poly. time.

4. Show that if $A \leq_P B$ and A is NP-hard, then B is NP-hard.

Take $X \in \text{NP}$, as A is NP-hard,
then

$$X \leq_P A$$

As $A \leq_P B$, then by
exercise 3 we have that

$$X \leq_P B$$

As X is arbitrary, B is
NP-hard.

To show that B is NP-complete,
we need $B \in \text{NP}$, which is
not guaranteed.

5. For each statement below, prove whether it is true or false.

(a) For every NP-hard problem X , we have $X \leq_P \text{SAT}$.

(b) Let X be an NP-complete problem. If there exists an algorithm to solve X in polynomial time, then SAT can be solved in polynomial time.

(a) FALSO

X is NP-hard, so $\forall L \in \text{NP}; L \leq_P X$.

SAT is in NP, and is NP-hard; so

$L \leq_P \text{SAT}$ and $\text{SAT} \leq_P X$.

But we want $X \leq_P \text{SAT}$

To prove that $X \leq_P \text{SAT}$, we need

$X \in \text{NP}$, which is not guaranteed because there are NP-hard problems that are not in NP.

(b) TRUE

X is NP-complete, so $X \in \text{NP}$ and $\forall L \in \text{NP}; L \leq_P X$. Since $\text{SAT} \in \text{NP}$, $\text{SAT} \leq_P X$.

But $X \in P$, so there exists cl that decides X in polynomial time. As $\text{SAT} \leq_P X$;

$w \in \text{SAT} \iff f(w) \in X$.

so SAT can be solved in TIME by using f and then deciding with cl .

P6)

Let M be a Turing machine and t a number in unary, let's consider the following problem

$$\text{ACCEPTS@}t = \{ \langle M, t \rangle : M \text{ accepts some input in less than } t \text{ steps} \}$$

Prove (by definition) that this problem is NP-complete

① Prove that $\text{ACCEPTS@}t$ is in NP.

a certificate for $\text{ACCEPTS@}t$ is given by the string that M accepted by M at time less than t .

note: this string is of size at most t . (Why?)

given input $\langle M, t \rangle$ and certificate c , the verifier can simulate M for $t-1$ steps and check whether M accepts (or not) the input.

this is done in polynomial time w.r.t the encoding of M and t .

∴ $\text{ACCEPTS@}t \in \text{NP}$.

② Prove that $\text{ACCEPTS@}t$ is NP-hard

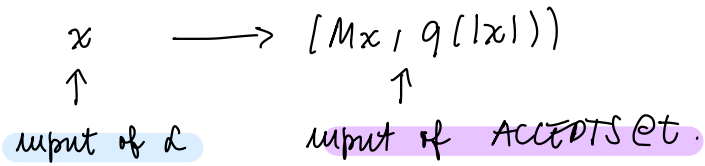
lets first recall the definition of NP-hard

Definition 3 (NP-Hardness) A language B is NP-hard if for any $A \in \text{NP}$ we have $A \leq_P B$.

We need to prove that every language in NP reduces to $\text{ACCEPTS@}t$.

Let $L \in \text{NP}$ and V_L a verifier for L .

As $L \in \text{NP}$, $V_L(x, c)$ halts in TIME in $|x|$ for all certificate c . Now we do the mapping.



where

$Mx :=$ machine that simulates $V_L(x, c)$ on input c

$q(\cdot) :=$ a polynomial that bounds the runtime of Mx $\forall x \in L$
 $|x| = m$

Clearly such polynomial $q(\cdot)$ exists because M_x runs in polynomial time w.r.t $|x|$ (why?)

the mapping described can be computed in polynomial time w.r.t $|x|$.

now, let's check if the reduction is correct, this is,

$$\underbrace{x \in L}_{(1)} \iff \underbrace{\langle M_x, q(|x|) \rangle}_{(2)} \in \text{ACCEPTS}@t.$$

we will prove both ways at the same time
then by definition of a verifier

$$x \in L \iff \exists c. V(x, c) \text{ accepts.}$$

now, by construction of M_x and q .

$$\exists c. V(x, c) \text{ accepts}$$

$$\iff M_x \text{ accepts } c \text{ } (|c| \leq q(|x|)) \text{ in time } q(|x|)$$

$$\iff \langle M_x, q(|x|) \rangle \in \text{ACCEPTS}@t.$$